Agentic Artificial Intelligence (AI): Architectures, Taxonomies, and Evaluation of Large Language Model Agents

Arunkumar V
University College of Engineering, Anna University
Tiruchirappalli, Tamil Nadu, India
arunkumarv1530@gmail.com

Gangadharan G.R.
National Institute of Technology
Tiruchirappalli, India
ganga@nitt.edu

Rajkumar Buyya
School of Computing and Information Systems
University of Melbourne, Australia
rbuyya@unimelb.edu.au

Abstract

Artificial Intelligence is moving from models that only generate text to Agentic AI, where systems behave as autonomous entities that can perceive, reason, plan, and act. Large Language Models (LLMs) are no longer used only as passive knowledge engines but as cognitive controllers that combine memory, tool use, and feedback from their environment to pursue extended goals. This shift already supports the automation of complex workflows in software engineering, scientific discovery, and web navigation, yet the variety of emerging designs, from simple single loop agents to hierarchical multi agent systems, makes the landscape hard to navigate. In this paper, we investigate architectures and propose a unified taxonomy that breaks agents into Perception, Brain, Planning, Action, Tool Use, and Collaboration. We use this lens to describe the move from linear reasoning procedures to native inference time reasoning models, and the transition from fixed API calls to open standards like the Model Context Protocol (MCP) and Native Computer Use. We also group the environments in which these agents operate, including digital operating systems, embodied robotics, and other specialized domains, and we review current evaluation practices. Finally, we highlight open challenges, such as hallucination in action, infinite loops, and prompt injection, and outline future research directions toward more robust and reliable autonomous systems.

Keywords: Agentic AI, Large Language Models, Autonomous Agents, Multi-Agent Systems, Cognitive Architectures, Tool Use, Planning.

Impact Statement

Agentic AI changes the role of AI systems from conversation partners to active collaborators that can carry out tasks end to end. This paper investigates the architectures that let Large Language Models (LLMs) run complex workflows in software engineering, scientific discovery, and robotics, and explains how the field is moving from single agent loops to organized multi agent systems. We also call out key risks, including prompt injection and hallucination in action, and offer a practical roadmap for building autonomous systems that are robust, secure, efficient, and suitable for open ended real world environments.

1 Introduction

The field of Artificial Intelligence is moving from "Generative AI", which focuses on mapping inputs to static outputs, to Agentic AI, where systems are designed to actively change the state of their environment through perception, reasoning, and action. This shift is no longer driven only by prompt engineering.

Frontier model families now expose stronger native reasoning behaviors, including configurable inference time reasoning budgets in dedicated reasoning models, which changes how planners and controllers are built in practice [29, 31]. At the same time, general purpose foundation models such as the OpenAI GPT 5 family, Google Gemini 3 Pro, and the Claude 4.5 family increasingly support structured tool use and multimodal interaction, yet in their base chat form they still operate turn by turn and do not maintain robust task state or permissions across long horizons [97–99].

The idea of an autonomous agent has a long history in computer science. Classical work [6] treated agents as encapsulated entities with situated autonomy, built on symbolic logic and hand written rules. Modern LLM based agents differ in important ways from both symbolic systems and reinforcement learning agents. Instead of depending on fixed symbolic representations or task specific policies learned through repeated interaction with an environment, contemporary agents draw on the probabilistic world knowledge encoded in foundation models and can generalize to unseen tasks with minimal task specific training, often achieving zero shot transfer [7]. In practice, this allows agents to operate in open ended domains, from fixing bugs in large software repositories [8] to designing scientific workflows that combine hypothesis generation, code execution, and result interpretation [9].

The motivation for agentic systems is driven by workflows that cannot be completed within the context window, reliability envelope, or tool permissions of a single model call. However, the engineering response is not simply to make agents more autonomous. A major practical shift is toward controllable orchestration, where developers specify explicit state transitions and guardrails, and models fill in local decisions. This is visible in graph based orchestration frameworks and state machines that prioritize debuggability, checkpointing, and human approvals, often described as flow engineering [32, 33]. These orchestration choices interact directly with reliability and safety because the controller determines what actions are possible, when the agent can loop, and where verification and escalation occur.

Deploying Agentic AI also introduces risks that do not arise for static LLMs. Once a model can execute actions such as modifying files, running code, or operating a desktop interface, hallucinations can become concrete failures rather than incorrect text. Security threats also expand because agents must ingest untrusted content and act on it. Indirect prompt injection is a central example, where malicious instructions are embedded in web pages, documents, or tool outputs and then followed by an instruction obedient agent [139]. These risks are amplified by new interaction modes such as native computer use, where agents operate user interfaces through screenshots and mouse and keyboard actions [37, 38]. In parallel, tool integration is becoming more standardized at the infrastructure layer. The Model Context Protocol provides a common way to expose tools and resources to agents, reducing fragmentation in connector schemas and enabling governance patterns such as allowlists and audit logging at the protocol boundary [34, 35].

Contributions of this paper

Existing surveys on LLM based agents and agentic AI have mainly taken four complementary perspectives: (i) broad overviews of LLM based autonomous agents and their applications [18, 63], (ii) conceptual and definition oriented treatments of agentic AI that go beyond LLMs [5], (iii) methodology focused surveys centered on tool use, planning, and feedback learning [17, 24], and (iv) recent reviews that describe LLM agents mainly from a methodology point of view [22]. In contrast, this survey is explicitly architecture and engineering focused: we start from a formal POMDP based agentic control loop and organize the literature around how concrete systems are built, deployed, and evaluated in practice, including inference time reasoning, controllable orchestration, and standardized tool connectivity.
Concretely, our main contributions are:

- **Unified architecture focused taxonomy.** We propose a unified taxonomy that decomposes LLM based agents into six modular dimensions: Core Components (perception, memory, action, profiling), Cognitive Architecture (planning, reflection), Learning, Multi Agent Systems, Environments, and Evaluation. This architecture first view connects each component to the underlying control loop and

complements prior surveys that group work mainly by applications [18, 63] or by paradigms such as tool use, planning, and feedback learning [17].

- **Engineering and systems perspective.** Beyond high level methodology, we highlight concrete design choices that matter in deployed systems: memory backends and retention policies, agent computer interfaces and computer use actions, the shift from JSON style function calling to code as action, standardized connector layers such as MCP, and orchestration controllers that enforce typed state and explicit transitions. Compared to methodology centered surveys [22], our focus is on how these pieces are assembled into robust, monitorable agent systems.
- **From autonomous loops to controllable graphs.** We give a unified treatment of multi agent systems that links classical MAS ideas with modern LLM based frameworks. We describe interaction patterns such as chain, star, mesh, and explicit workflow graphs, and we analyze frameworks including CAMEL, AutoGen, MetaGPT, LangGraph, Swarm, and MAKER under the same lens. This extends earlier multi agent surveys that often discuss collaboration mechanisms in isolation [81, 82].
- **Holistic view of environments, evaluation, and safety.** We place evaluation directly in the architectural space using the CLASSic dimensions of cost, latency, accuracy, security, and stability. We link architectural choices such as hierarchical planning, code execution, graph controllers, and computer use actions to concrete failure modes including hallucination in action, infinite loops, and prompt injection. We also summarize lessons from enterprise oriented benchmarks and deployment studies [4, 75, 114].

Among existing works, Wang *et al.* [18] and Xi *et al.* [63] provide broad overviews of LLM based autonomous agents but devote comparatively less attention to detailed engineering of perception modules, memory backends, connector standards, and controllable orchestration. Luo *et al.* [22] survey the field through a methodology centered taxonomy focused on planning, tool use, learning, and collaboration. Our taxonomy is complementary: we derive an explicit systems architecture that covers perception, memory, the agent brain, and action, and we use this to guide how to build robust agents rather than only cataloging what existing agents can do.

The rest of the paper is organized as follows. Section II provides formal definitions and background. Section III presents the taxonomy of the Agentic AI ecosystem. Section IV details the unified architecture of Agentic AI. Section V analyzes Multi Agent Systems and their interaction topologies. Section VI discusses Environments and Applications where the agents operate. Section VII addresses Evaluation and Safety assessment. Section VIII outlines Challenges and Future Directions of the Agentic AI ecosystem, followed by concluding remarks in Section IX.

2 Background and Definitions

This section outlines the evolution of the agent concept, formalizes the definition of an agent using a mathematical decision-making framework, and presents a holistic taxonomy that constitute a modern agentic system.

2.1 From Symbolic and RL Agents to LLM based Agents

The concept of an "agent" has changed substantially over the history of computer science. Early work focused on symbolic agents, as described in foundational studies such as [6], which relied on predefined rules, formal logic, and fixed constraints to operate in closed environments. These systems were often robust within their design envelope but brittle when confronted with situations that fell outside their programmed rules. As research progressed, attention shifted toward Reinforcement Learning (RL) agents, where policies are learned through repeated interaction with an environment and through explicit trial and error. These agents are very effective on specific control tasks, yet they usually lack the generalization

ability needed for open ended reasoning or flexible natural language interaction and they demand large amounts of data and careful sample efficiency for each new task [15].

The modern LLM based agent can be seen as a third paradigm. Instead of depending on hard coded rules or narrowly defined reward functions, these agents use a pretrained Large Language Model as a general purpose cognitive controller. In this view, the LLM acts as a reasoning engine that can be augmented with external memory and execution modules [16], and the design focus shifts from training policies to prompt design, tool integration, and orchestration. This allows agents to transfer the semantic knowledge learned from large internet corpora into concrete action oriented tasks [17]. In practice, such agents can often generalize in a zero shot manner to new environments that include both software development and robotics, achieving behaviors that were not feasible with purely symbolic systems or with standard RL agents alone.

2.2 Formal Definition: The Agentic Control Loop

Although concrete architectures differ, there is growing agreement on how to describe the basic mathematical structure of an LLM based agent. Rather than viewing an agent only as a policy, it is useful to treat it as a dynamic control system that operates within a Partially Observable Markov Decision Process (POMDP). We write the agent system $\mathcal{A}$ as a tuple

$$\mathcal{A} = \langle \mathcal{S}, \mathcal{O}, \mathcal{M}, \mathcal{T}, \pi \rangle,$$

where $\mathcal{S}$ is the state space, $\mathcal{O}$ the observation space, $\mathcal{M}$ the internal memory space, $\mathcal{T}$ the action or tool space, and π the policy.

At each discrete time step t, the system moves through a cycle of linked functions.

2.2.1 Perception Function (Φ)

The agent does not have access to the full environment state $S_t \in \mathcal{S}$. Instead, it receives a partial observation O_t through a perception function Φ, which may include multimodal encoders (for example vision models such as CLIP) or text based wrappers:

$$O_t = \Phi(S_t) \quad \text{with} \quad O_t \subset S_t. \tag{1}$$

2.2.2 Memory Update Mechanism (μ)

In contrast to stateless reinforcement learning agents, LLM based agents maintain a mutable internal state M_t. This state is updated by a function μ that combines the new observation O_t, the previous reasoning trace Z_{t-1}, and the execution feedback E_{t-1}:

$$M_t = \mu(M_{t-1}, O_t, Z_{t-1}, E_{t-1}). \tag{2}$$

This update step subsumes retrieval augmented generation (RAG) [16], where μ selects and injects relevant information from the agent's long term memory into the current context.

2.2.3 Cognitive Planning (Ψ)

A defining feature of Agentic AI is the latent reasoning step Z_t (the thought or plan) that is produced before any external action is taken. We model this as a probabilistic inference process parameterized by the LLM θ:

$$Z_t \sim P_\theta(Z_t \mid M_t, O_t). \tag{3}$$

The variable Z_t can represent a simple chain of thought or a more structured hierarchical plan. In advanced architectures such as RAP [19], this planning phase is implemented as a recursive tree search over possible future trajectories.

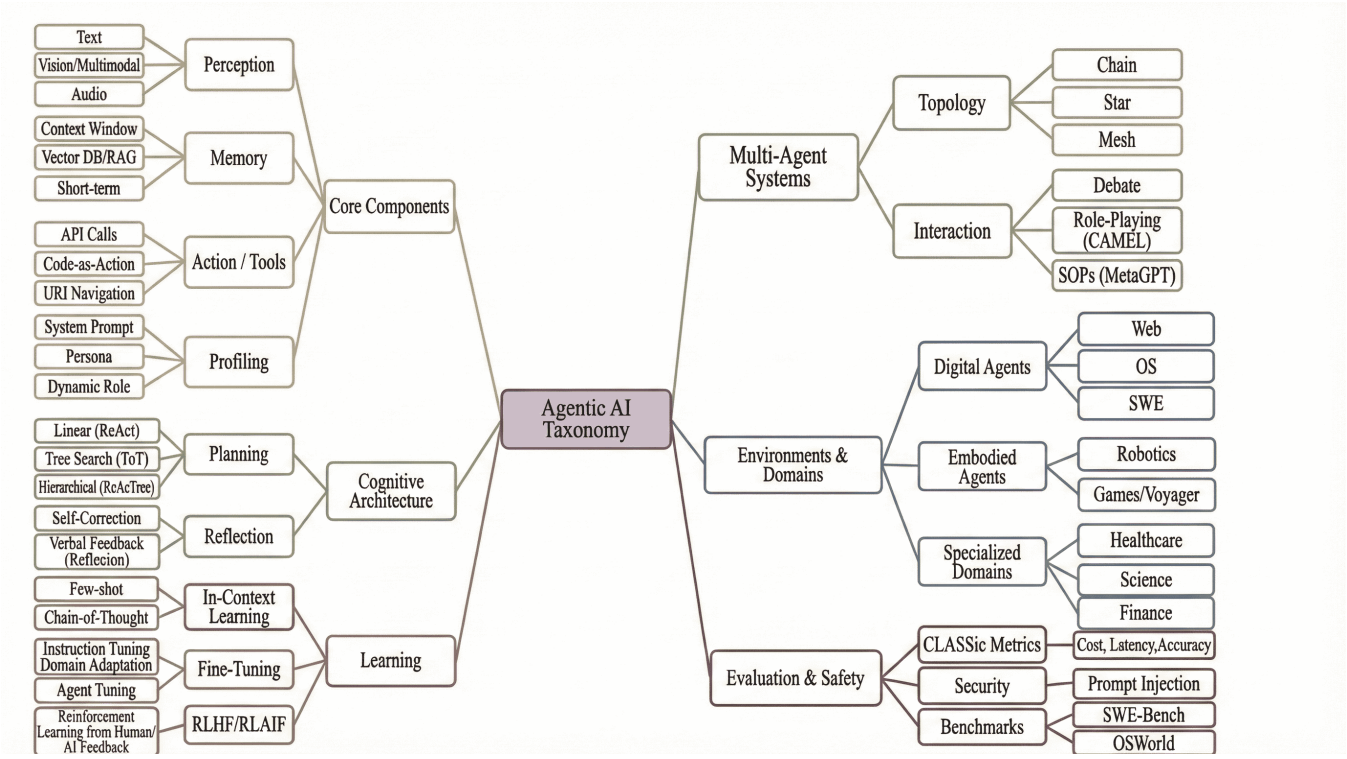

Figure 1: **Taxonomy of the Agentic AI ecosystem.** The figure organizes the literature into six main dimensions: Core Components, Cognitive Architecture, Learning, Multi Agent Systems, Environments, and Evaluation. Together, these dimensions trace the field's progression from simple text based loops to complex hierarchical systems that can operate in open ended environments.

2.2.4 Action Policy (π) and Execution

Finally, the agent chooses and executes an action A_t from the tool space $\mathcal{T}$. The policy π is conditioned directly on the reasoning trace, so that actions are explicitly grounded in the internal plan:

$$A_t \sim \pi_\theta(A_t \mid Z_t, M_t). \tag{4}$$

The environment then reacts to this action through a state transition $S_{t+1} \leftarrow \text{Env}(S_t, A_t)$ and produces feedback E_t, which flows back into the next perception and memory update steps, closing the control loop [20].

3 Taxonomy

3.1 Holistic Taxonomy of the Agentic AI Ecosystem

Fig. 1 gives a high level view of the agentic AI ecosystem and the way its architectures have evolved over time. The field has grown by expanding along six connected dimensions, moving from individual capabilities inside a single model to systems that coordinate many agents and include explicit mechanisms for evaluation and safety.

Core Components The foundation of any agent is its interface with the world. Perception has moved from processing only raw text to handling visual, multimodal, and audio inputs, which allows agents to interpret complex graphical user interfaces. Memory has shifted from short lived context windows to persistent storage backed by vector databases and retrieval augmented generation (RAG), which supports

continuity over long horizons. In the same spirit, action interfaces have developed from fixed API calls into more flexible Code as Action and direct URI navigation. Profiling completes this layer by defining the agent's identity through system prompts and dynamic roles, so that its behavior remains consistent across different tasks.

Cognitive Architecture The cognitive architecture dimension describes how agents reason. Early systems relied on linear planning loops such as ReAct. To deal with more complex problems, recent work has adopted hierarchical structures that use tree search methods, for example Tree of Thoughts, and recursive decomposition as in ReAcTree. To improve reliability, these planners are complemented by reflection mechanisms, including self correction and verbal feedback methods such as Reflexion, which allow agents to critique and refine their plans before they act.

Learning The learning dimension captures how agents acquire and improve capabilities over time. At one end of the spectrum are in context methods, such as few shot prompting and Chain of Thought, which are temporary and live entirely in the prompt. At the other end are permanent weight updates through fine tuning, including instruction tuning and agent specific tuning. On top of these, alignment techniques such as RLHF and RLAIF adjust agent behavior using feedback from humans or from other models, and are increasingly used to shape decision making in realistic settings.

Multi Agent Systems As tasks grow beyond the capacity of a single model, the taxonomy extends to multi agent systems. This dimension distinguishes between interaction styles that range from adversarial debate to cooperative role playing, as in CAMEL, and structured workflows based on standard operating procedures, as in MetaGPT. It also highlights different communication topologies, where agents can be organized in chains for sequential processing, in star shaped configurations with a central coordinator, or in mesh like swarms for more decentralized collaboration.

Environments and Domains Agents are also defined by the environments in which they operate. The taxonomy groups these into digital agents that work inside web browsers, operating systems, or software engineering tools, embodied agents in robotics and games such as Voyager, and specialized domains including healthcare, science, and finance. Each class of environment comes with its own constraints and affordances, which in turn impose specific requirements on perception, memory, and action design.

Evaluation and Safety The final dimension addresses how agentic systems are assessed and secured. Evaluation frameworks have moved beyond single accuracy scores and now incorporate the CLASSic metrics of cost, latency, accuracy, security, and stability. Security research focuses in particular on defending against threats such as prompt injection. At the same time, standardized benchmarks such as SWE-Bench for software engineering and OSWorld for operating system control provide shared references for tracking progress across different architectures.

We use this taxonomy as the organizing structure for the remainder of the paper. It allows us to analyze each dimension in depth, from single agent cognitive architectures to multi agent coordination patterns and the evaluation and safety practices that are required for real world deployment.

4 The Unified Architecture of Agentic AI

We view the architecture of an autonomous agent as a cognitive pipeline that turns perception into action through a central decision making process. As illustrated in the feedback loop in Fig. 2, this system integrates modular resources with a reasoning engine that runs over time. In line with the taxonomy in Fig. 1, we break this pipeline into three layers: i) the Core Components, which provide interfaces for perception, memory, action, and profiling; ii) the Cognitive Architecture, which carries out planning and reflection; and iii) Learning, which describes how agents acquire and refine their capabilities.

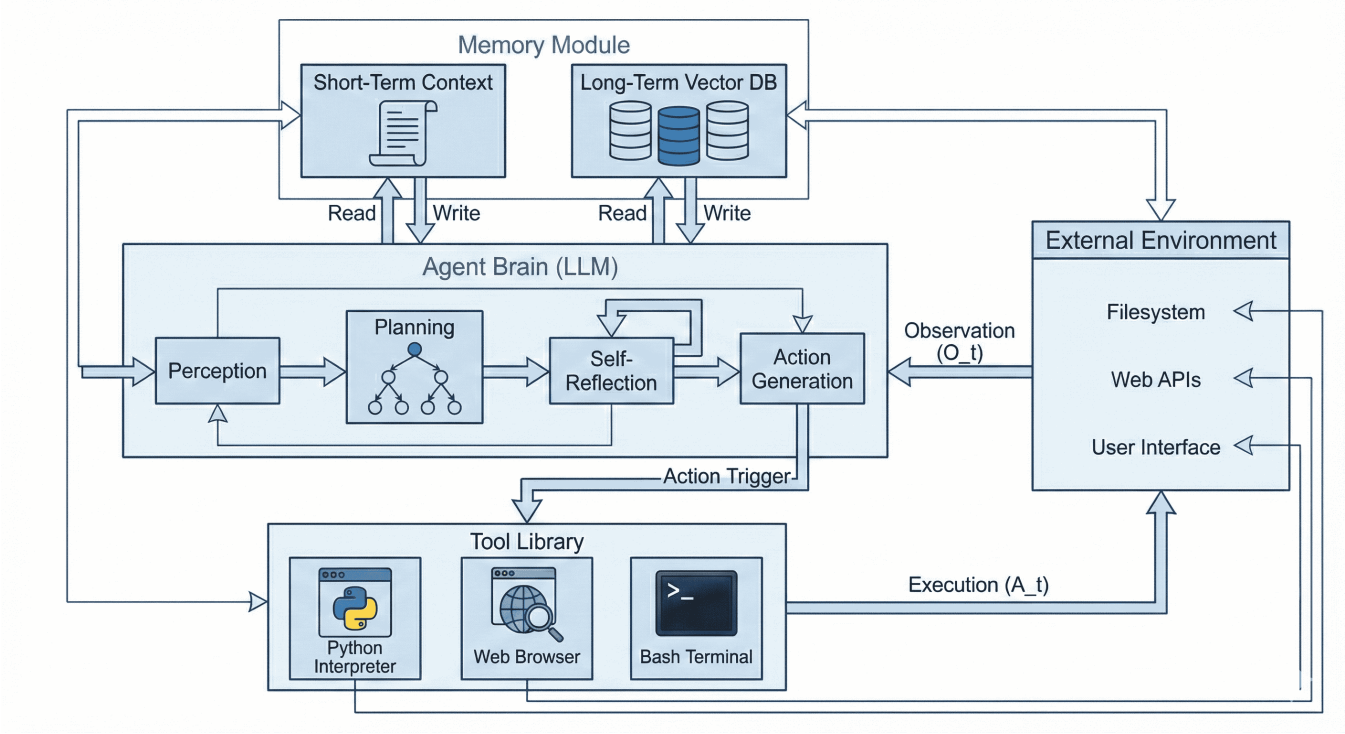

Figure 2: **The unified architecture of Agentic AI.** The system is shown as a modified POMDP loop. The agent brain at the center transforms each observation (O_t) into a reasoning trace (Z_t) using hierarchical planning and self reflection. A dual stream memory module at the top supports context retrieval, while a tool library at the bottom executes code based actions (A_t) that change the external environment on the right.

Table 1: **Evolution of agent perception modules.** The table summarizes how agents ground themselves in digital and physical environments and highlights the move from pure text processing to multimodal vision and audio encodings.

Model / Framework	Input modality	Grounding mechanism	Target environment	Critical challenge
WebVoyager [108]	Vision (screenshots)	CLIP based visual encoder	Open web browsers	*Visual clutter*: distracted by ads and pop ups; high token cost for images.
AppAgent [91]	Vision + touch coordinates	Visual to action mapping	Smartphone apps (Android)	*Dynamic UIs*: struggles with animations or fading elements.
SeeAct [92]	Hybrid (HTML + vision)	Cross attention	Web forms	*Grounding gap*: hallucinates (x, y) coordinates for small buttons.
Magma [156]	Video + proprioception	Set of Mark (SoM)	Robotics	*Latency*: real time video processing is too slow for reactive control.
AudioGPT [93]	Audio and speech	Audio foundation models	Voice assistants	*Turn taking*: difficulty handling interruptions in live conversation.
3D LLM [94]	3D point clouds	Spatial encoders	Embodied navigation	*Spatial resolution*: low fidelity in detecting small obstacles.

4.1 Core Components

Modern agent architectures are built from modular components coordinated by a cognitive process. Building on infrastructure layers [21] and construction taxonomies [22], we write the core components of an agent A as a tuple

$$A = \langle \Phi, M, T, P \rangle,$$

where Φ is perception, M is memory, T is the set of actions and tools, and P is profiling.

4.1.1 Perception

Perception (Φ) is the interface between the agent and its environment. Early agents such as ReAct operated entirely in the text domain. In contrast, recent systems make use of multimodal large language models (MLLMs) that ground their reasoning in higher dimensional sensory inputs.

Table 2: **Evolution of agentic memory architectures.** We compare approaches for long term state persistence. The retention strategy column indicates how each system prevents uncontrolled growth of context, for example by forgetting, summarizing, or paging.

Architecture	Memory structure	Retrieval strategy	Retention strategy	Primary benefit
Generative Agents [59]	Natural language stream	Scoring by recency and relevance	Reflection and summarization	Social consistency and long horizon coherence in simulations
MemoryBank [61]	Hierarchical clusters	Hierarchical traversal	Exponential decay	Keeps memory relevant through forgetting
ChatDB [62]	Symbolic SQL tables	SQL queries	Exact storage	High precision for structured and numerical state
MemGPT [102]	Paged long term memory	Controller driven retrieval	Explicit paging and summarization	Manages long contexts through externalized memory control
MemInsight [76]	Insight level summaries	Semantic clustering	Merge and compress	Converts episodic traces into compact semantic memories
MemAgent [77]	Active read and write store	Policy learned retrieval	Policy driven pruning	Learns what to store, summarize, and discard across sessions

Multimodal grounding for digital and embodied settings Modern perception modules move beyond text-only inputs to support UI-level grounding (screenshots, touch coordinates, and hybrid DOM+vision) and embodied sensing (video, audio, and 3D geometry). WebVoyager and AppAgent demonstrate screenshot-driven and coordinate-based control in browsers and mobile apps, but both expose a persistent grounding bottleneck: mapping high-level intent to precise UI targets, where small elements and dynamic layouts cause drift or hallucinated clicks [91, 92, 108]. For time-varying scenes, Magma extends perception to continuous video streams for robotic manipulation [156]. Beyond vision, AudioGPT integrates speech/audio understanding for voice-first interaction [93], while 3D-LLM injects point-cloud representations to preserve spatial affordances that are lost in 2D projections [94]. Table 1 summarizes representative designs, grounding mechanisms, and dominant failure modes.

4.1.2 Memory

To behave as persistent agents rather than one off sessions, systems need mechanisms that preserve state over time and support consistent behavior. Memory components address these needs and go beyond simple vector retrieval to include structure, retention policies, and explicit mechanisms for summarization and deletion.

Memory as retrieval, structure, and retention policy A central systems decision is how an agent manages working context under cost and attention constraints: long-context prompting scales poorly with irrelevant history, so retrieval-augmented generation remains important for focusing on task-relevant state [16]. MemoryBank shows that hierarchical summaries can outperform raw-log prompting by making retrieval structured and self-pruning [61], while Chain-of-Agents distributes extremely long contexts across coordinated workers when a single prompt is insufficient [78]. For long-horizon coherence, Generative Agents maintain a timestamped memory stream [59], and ACAN-style alignment improves recall beyond vanilla similarity retrieval [60]. Retention policies (summarize/forget/prune) prevent unbounded growth: MemInsight compresses episodic traces into semantic "insights" [76], and MemAgent treats memory management as a learned decision process across sessions [77]. When state is naturally structured, ChatDB provides high-precision symbolic memory via SQL [62], while MemGPT formalizes paging between short context and external stores under an explicit controller [102]. Table 2 compares these designs by structure, retrieval, and retention strategy.

4.1.3 Action and Tools

While the cognitive architecture (discussed below) provides the reasoning blueprint, the action component allows the agent to actually change its environment. This module turns the semantic intent produced by the planning layer into concrete operations. Over time, the space of possible actions has moved from fixed, predefined function calls toward more flexible code execution and interface level navigation. For clarity, we group these mechanisms into four main paradigms: API calls, Code as Action, Agent Computer Interfaces, and embodied Vision Language Action (VLA).

Table 3: **Evolution of agentic action spaces.** The table contrasts the main ways agents execute tasks. Recent production systems add computer use actions for generic GUI control, alongside code execution and constrained tool calls.

Paradigm	Action space	Representative frameworks	Key advantage	Primary limitation
API based	Predefined structured calls	Toolformer [52], Gorilla [57]	Safety through restricted scope	Statelessness and tool schema friction
Code as action	Executable scripts (often Python)	CodeAct [3], Voyager [64]	Rich control flow and state	Sandboxing risk and runtime errors
ACI based	Curated shell or IDE actions	SWE agent [8]	Efficient context use	Requires interface design and upkeep
Computer use actions	Mouse, keyboard, screenshots	Claude computer use [36], Operator [38]	Works on arbitrary GUIs without app APIs	Latency, brittleness, and injection risk
Embodied VLA	Continuous motor primitives	Gemini Robotics [118]	Physical grounding	Latency and sim to real gaps

From constrained APIs to code, interfaces, and embodied control Agent action spaces have expanded from constrained API/function calls to executable programs, UI-level control, and continuous embodied policies. API-centric systems such as Toolformer and Gorilla emphasize safety and tool selection through structured calls, but often require brittle schema prompting and ad hoc state passing across steps [52, 57]. At the infrastructure layer, MCP standardizes tool discovery and invocation so controllers can connect to evolving tool catalogs with governance boundaries such as allowlists, authentication, and audit logging [34, 35].

In parallel, "code as action" uses executable scripts as the control interface—enabling variables and control flow and aligning better with code-heavy pretraining distributions—illustrated by CodeAct and Voyager's reusable skill library [3, 64]. For long-horizon digital work, curated Agent Computer Interfaces (e.g., SWE-agent) reduce context load by exposing simplified, agent-friendly shells/IDEs [8]. Finally, computer-use actions (screenshots + mouse/keyboard) enable generic GUI operation but introduce latency and broader injection risk [36, 38], while embodied VLA systems push toward direct perception-to-motor control in robotics [118]. Table 3 summarizes these paradigms and their tradeoffs.

4.1.4 Profiling

Memory gives an agent continuity of experience, while profiling gives it a stable character. This module specifies the identity, role, and implicit constraints of the agent [23]. In practice, profiling corresponds to the system prompt or persona that shapes behavior, for example, "You are a senior Python engineer who writes clear and well tested code." Such profiles narrow the search space and improve alignment. An agent whose profile emphasizes security will naturally favor safer plans than one that prioritizes speed. More advanced designs allow for dynamic roles, so that an agent can switch, for instance, from a writer role to an editor role as a task moves from drafting to revision.

4.2 Cognitive Architecture

The cognitive architecture is the agent's decision core: it decomposes goals into action sequences A_t and monitors execution for inconsistency and failure. Unlike classical symbolic planners with hand-written rules and explicit environment models, LLM-based agents leverage probabilistic world knowledge to propose plans and revise them under feedback. Following our taxonomy, we describe this layer through *planning* (how trajectories are constructed) and *reflection* (how trajectories are evaluated and improved).

4.2.1 Planning

A foundational agentic pattern is ReAct [1], which interleaves intermediate reasoning with environment interaction, in contrast to purely latent Chain-of-Thought prompting [25]. ReAct executes trajectories $\tau = \{o_1, z_1, a_1, o_2, z_2, a_2, \dots\}$, where rationales z_t condition actions a_t and subsequent observations, improving groundedness but remaining vulnerable to myopia and error propagation (e.g., early mistakes leading to unproductive loops).

To reduce myopia, many approaches add explicit branching and search. Tree of Thoughts treats intermediate thoughts as search nodes and explores alternatives via breadth/depth strategies [26]. LATS

Table 4: **Comparative analysis of agentic cognitive architectures.** We compare planning methodologies by topology, search strategy, and inference cost. *Token complexity* estimates inference overhead relative to a standard zero shot prompt (N: steps, b: branching factor, d: depth, B: inference time compute budget).

Method	Core mechanism	Reasoning topology	Token complexity	Key advantage	Critical limitation
Standard CoT [25]	Latent reasoning steps	Linear chain	Low ($\sim 1\times$)	Low latency	Error propagation without recovery
ReAct [1]	Interleaved thought and action	Linear loop	Medium ($N\times$)	Groundedness	Myopic behavior and looping
Reflexion [44]	Verbal reinforcement buffers	Cyclical loop	High ($k \cdot N\times$)	Self correction	Hallucinated critiques and context overflow
Tree of Thoughts [26]	External branching search	Tree structure	Very high ($b^d\times$)	Backtracking and global search	Latency and combinatorial growth
LATS [127]	MCTS with value feedback	Graph or tree	Very high	Strong accuracy on hard tasks	Depends on evaluators and heavy compute
Reasoning models (o1, o3) [28–30]	Internalized inference time search	Adaptive implicit tree	Variable ($\propto B$)	Compute quality tradeoff inside the model	Higher cost and limited transparency
ReAcTree [2]	Recursive sub agent spawning	Hierarchical tree	Medium high	Modular long horizon solving	State synchronization complexity

further connects agent planning to MCTS by using the model as both proposal and evaluator, enabling selection among candidates before committing to irreversible tool calls [127].

A recent shift is that frontier reasoning models internalize parts of search at inference time. OpenAI's o1 line emphasizes improved performance with increased test-time "thinking" [28, 29], aligning with work on test-time compute scaling [31] and subsequent releases such as o3 [30]. This does not remove external planning modules, but changes their role: modern systems increasingly combine controllable inference-time reasoning budgets with external controllers that enforce safety, state persistence, and tool permissions.

As task complexity grows, flat planners still face context and modularity limits, motivating hierarchical decomposition. ReAcTree distributes long-horizon planning across recursive sub-agents with explicit control flow [2], while GoalAct uses a two-tier structure with a global milestone planner and local executors [27]. These designs improve interpretability and fault isolation but can increase token and latency costs in large or noisy environments.

To improve efficiency without additional labels, test-time optimization methods refine behavior using signals extracted from agent trajectories (e.g., RISE and test-time self-improvement) [49, 50]. Proactive architectures further separate internal deliberation from user-facing outputs to bound interactive latency [51]. Table 4 positions these approaches alongside prompted search, hierarchy, and inference-time compute scaling.

4.2.2 Reflection

Reflection mechanisms allow agents to critique and adapt based on their own trajectories, transforming sparse success/failure signals into actionable guidance. A common pattern is *verbal reinforcement*: Reflexion stores natural-language critiques of failures and conditions future attempts on these lessons rather than updating weights [44]. Complementarily, *self-correction protocols* apply critique before committing to outputs. Self-Refine implements an iterative generate–critic–revise loop [65], while CRITIC reduces purely internal confirmation loops by requiring tool-interactive validation (e.g., interpreters or search) before accepting revisions [155].

Reflection is especially important under tool failures (timeouts, schema mismatches, repeated retries). PALADIN trains recovery behaviors on large collections of failure trajectories, enabling diagnosis and corrected retries that reduce loop-like failures [58]. Expel extracts reusable rules from past mistakes (e.g., safety or rollback heuristics) and applies them to new tasks, supporting both episode-level repair and cross-episode generalization [74].

4.3 Learning Paradigms

The learning dimension describes how agent capability improves over time, spanning ephemeral in-context adaptation, permanent weight updates, and non-parametric accumulation of executable skills. Early agents relied mainly on in-context learning (examples and heuristics in prompts), which is easy to deploy but short-lived and increasingly costly as prompts grow. This motivates agent tuning, where useful behaviors are internalized in weights via trajectory data: Agent-FLAN and FireAct fine-tune models on agent rollouts, and FireAct shows that "hot" trial-and-error trajectories can outperform prompt-only methods by shifting

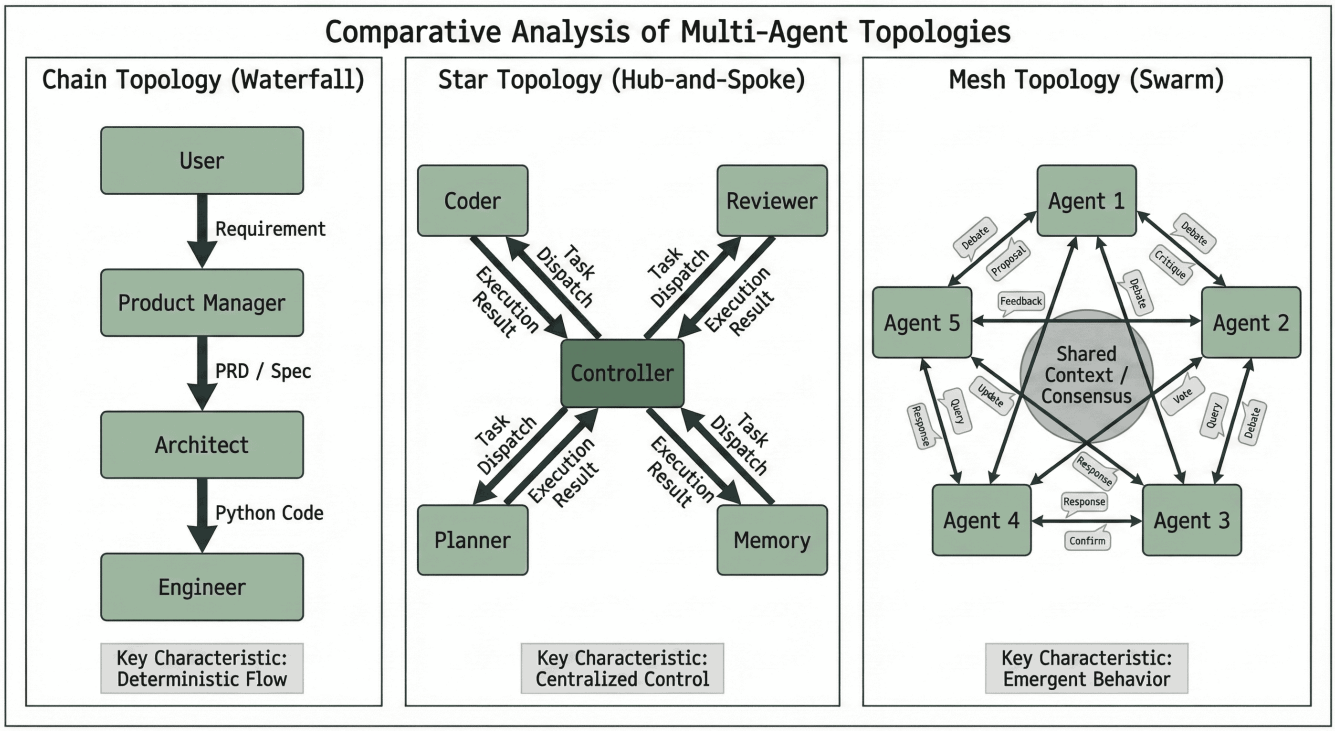

Figure 3: **Communication Topologies in Multi-Agent Systems.** We classify collaboration patterns into three dominant structures: (Left) Chain Topology, utilized by MetaGPT to enforce Standard Operating Procedures (SOPs) via sequential hand-offs; (Center) Star Topology, employed by AutoGen where a Controller agent dispatches tasks to specialized workers; and (Right) Mesh Topology, used in social simulations like Generative Agents to enable dynamic, unstructured interaction.

task logic from long prompts into compact parameters and reducing inference cost [67, 158]. However, effective tuning must avoid overfitting to surface formats rather than robust reasoning patterns.

Beyond supervised tuning, scalable oversight is a bottleneck. RLAIF uses AI feedback to generate preference labels [159], enabling process-level reward models such as AgentRM and AgentPRM to provide dense step-wise guidance without heavy human annotation [68, 69]. These signals support test-time search and selection among candidate trajectories, complementing hand-designed heuristics. Self-improvement pipelines further show that agents can bootstrap performance by mining failures and generating synthetic corrections, reporting sizable gains on web-agent tasks [66]. Finally, agents can improve without weight updates by storing and reusing executable skills: Voyager maintains an external skill library of successful code fragments that can be retrieved and composed, accelerating open-ended learning while avoiding catastrophic forgetting [64].

5 From Single to Multi Agent Systems

While single agent architectures have shown strong capabilities in isolation, many real world problems still exceed the context window, domain expertise, or cognitive capacity of a single model. This has led to a shift toward multi agent systems (MAS), in which groups of specialized agents collaborate to complete workflows. As illustrated in Fig. 3, the effectiveness of a multi agent system depends heavily on its interaction topology. Following [81], these structures are usually grouped into three main patterns: i) chain or waterfall, where tasks are passed along a fixed sequence of agents for rigid workflows; ii) star or hub and spoke, where a central controller coordinates specialized workers; and iii) mesh or swarm, where agents interact in a more decentralized and dynamic way, for example during brainstorming.

Table 5: **Analysis of multi agent collaboration frameworks.** We map prominent frameworks to the coordination structures in Fig. 3 and include graph based orchestration as an increasingly common production pattern.

Framework	Topology	Role structure	Communication style	Conflict resolution	Ideal use case
CAMEL [79]	1 to 1 mesh	Symmetric	Inception prompting	None (chat loop)	Ideation and brainstorming
AutoGen [80]	Star or mesh	Dynamic	Message passing	Human in the loop	Prototyping heterogeneous tools
MetaGPT [11]	Chain (waterfall)	Static roles	SOP documents	Sequential handoff	Complex software engineering
ChatDev [84]	Chain	Static phases	Waterfall handoffs	Reviewer rejection	End to end app generation
DyLAN [126]	Dynamic graph	Dynamic selection	LLM optimized routing	Importance scoring	Focused reasoning collaboration
LangGraph [32]	Workflow graph	Developer defined	State machine execution	Guard nodes and approvals	Production flow engineering
Swarm [33]	Star with handoffs	Lightweight specialists	Handoff routines	Controller selection	Controllable coordination patterns
TradingAgents [87]	Mesh (market)	Heterogeneous	Auction or debate	Market equilibrium	Economic simulation
MAKER [85]	Hierarchical	Supervisor and worker	Cross examination	Verifier agent	High reliability logic tasks

5.1 Graph based orchestration and flow engineering

A key industry shift is the move from open ended multi agent chat loops toward explicit workflow graphs. Rather than relying on a generic manager agent to decide what to do next in free form dialogue, many production systems model the workflow as a state machine where nodes represent tool calls or LLM invocations and edges represent permissible transitions. This approach is often described as flow engineering, since the developer designs the control structure and the agent fills in local decisions within that structure.

LangGraph is a representative framework that operationalizes this idea by treating agent execution as graph traversal with explicit state persistence, checkpoints, and controlled cycles [32]. This makes long horizon behavior more debuggable and easier to align with organizational constraints, because developers can insert guard nodes, approval steps, and typed state updates at specific points in the graph.

OpenAI Swarm provides a complementary view that emphasizes lightweight agent handoffs and routines for orchestrating specialist behaviors, and it is positioned as a reference implementation for controllable coordination patterns [33]. In practice, these patterns align with a broader trend: orchestration is increasingly specified as an explicit controller layer, while the LLM focuses on local reasoning, tool parameterization, and recovery. Graph based designs also interact naturally with safety and evaluation, since the graph boundary defines what actions are even possible, which can reduce the frequency and severity of runaway loops.

5.2 Architectures of Collaboration and Role Playing

The core mechanism that drives many multi agent systems is role playing. The CAMEL framework [79] was an early and influential example. It introduced inception prompting to set up autonomous cooperative dialogues by assigning specific personas (for instance a stock trader) and starting a role flip conversation. CAMEL showed that agents could guide one another through multi step tasks using only these role prompts. However, CAMEL uses a simple one to one mesh topology that can drift into unproductive loops because there is no central decision maker. AutoGen [80] generalizes this idea by treating agents as conversable entities that can be wired together in more flexible ways. Developers can specify custom interaction graphs, and many practical setups adopt a star topology in which a user proxy or manager agent delegates subtasks to tools and worker agents and then aggregates their outputs. This structure also supports human in the loop oversight, which is critical for safety sensitive applications. Moving beyond fixed graphs, DyLAN [126] proposes a dynamic agent network. Instead of using a static interaction pattern, DyLAN estimates the contribution of each agent at every step and selects collaborators based on an importance score. Agents that are not helpful for the current problem are muted, which reduces token usage and cost, while relevant agents remain active. This adaptive routing leads to more efficient and focused collaboration during complex reasoning chains.

5.3 Organizational Metaphors: The Chain Topology

A distinct sub-class of multi-agent systems draws inspiration from human corporate structures, implementing Standard Operating Procedures (SOPs) to streamline collaboration.

5.3.1 Software Swarms

MetaGPT [11] formalized this approach by encoding SOPs directly into agent prompts. By assigning specific roles such as Product Manager, Architect, and Engineer, MetaGPT forces agents to generate standardized deliverables (e.g., PRDs, API Designs) that serve as strict inputs for the next agent in the waterfall. This effectively functions as an "operating system" for agents, reducing the hallucination rate by transforming unstructured chat into a rigorous workflow. This concept was further exemplified in ChatDev [84], which simulates an entire software company where agents self-organize into design, coding, and testing phases, achieving a 30% reduction in bugs compared to single-agent coding.

5.3.2 Hierarchical Verification

Linear handoffs can propagate errors if an upstream agent fails. To address this, MAKER [85] advances the organizational metaphor by introducing a "cross-examination" phase. MAKER demonstrates that by decomposing tasks into granular steps and using distinct "Verifier" agents to challenge the output of "Worker" agents, systems can execute million-step reasoning chains with near-zero error accumulation. This aligns with findings from BOLAA [86], which proved that orchestrating multiple smaller, specialized agents often outperforms a single massive model by distributing the cognitive load. This hierarchical safety extends to supervisory architectures. OVON [88] and The Good Parenting framework [89] propose "Parenting" topologies, where a "Reviewer" agent holds a distinct system prompt focused solely on critique and safety guidelines. Empirical results show that filtering "Child" agent outputs through a Supervisor can reduce hallucination rates by up to 100% in controlled environments.

5.4 Social Simulation and Debate: The Mesh Topology

While chains optimize for efficiency, Mesh Topologies optimize for creativity and diversity. In these systems, interaction is decentralized, allowing emergent behaviors to arise from agent-to-agent dynamics.

5.4.1 Adversarial Debate

Dialectical interaction has been proven to enhance reasoning performance. Research on Multiagent Debate [90] revealed that allowing multiple LLM instances to propose conflicting answers and critique each other's reasoning leads to a convergence on truth. This "society of minds" approach is further refined in Adaptive Debate [95], where agents assume specialized adversarial roles (e.g., "Devil's Advocate") to stress-test solutions. Recent work extends this to Communication Games [96], where agents play referential games to align on descriptions of complex molecules. This constrained communication forces agents to develop precise, compositional language, improving generalization to novel structures.

5.4.2 Economic and Social Simulation

Mesh topologies are also used to model complex systems. TradingAgents [87] replicates a financial market where specialized agents (Risk Managers, Technical Traders) debate investment decisions. The study reveals that organizational diversity leads to emergent market phenomena, such as price discovery, which a single agent cannot simulate. On a larger scale, Generative Agents [59] demonstrated how information diffuses through a population of agents in a simulated town. Recent work like SocioVerse [103] expands this to 10,000+ agents to study social norm propagation. These "World Models," including Genie 3 [104] and PAN [105], allow agents to simulate the consequences of actions in a physics-aware latent space before executing them in reality, bridging the gap between internal reasoning and external simulation.

Table 5 presents the state-of-the-art multiagent collaboration frameworks, contrasting their structural rigidity and conflict resolution mechanisms.

6 Environments and Applications

An agent is defined not only by its internal architecture but also by the environment in which it operates. Grounding the reasoning capabilities of Large Language Models into actionable execution requires distinct interfaces for different domains. We classify these environments into Digital (Web, OS, Enterprise), Embodied (Robotics), and Scientific domains.

6.1 Digital Agents: The Web, Operating Systems, and Enterprise

Most modern knowledge work occurs in digital interfaces, making web and desktop automation a central frontier. The field has moved from passive retrieval toward active execution in open-ended environments.

6.1.1 The Web Agent Evolution

Web agency is challenging primarily because interfaces vary widely and evolve dynamically. Mind2Web and WebArena established realistic long-horizon benchmarks across functional site clones (e.g., GitLab, e-commerce) [106,107]. Purely text-based agents often break on dynamic DOM changes or canvas elements, motivating multimodal approaches such as WebVoyager, which uses screenshots to infer visual UI structure [108]. However, visual agents introduce new failure modes: Environmental Distractions shows that models can over-attend to irrelevant UI elements (ads, pop-ups), causing action errors and motivating robustness methods such as task-aware cropping and attention masking [109].

Native computer use beyond DOM parsing Computer-use interfaces push this further by treating the desktop/browser as pixels plus low-level input actions. Anthropic's computer-use tooling for Claude exposes cursor movement, clicking, typing, and reading screen state [36, 37], reducing dependence on brittle DOM extraction and app-specific wrappers. OpenAI's Operator similarly targets GUI operation from screenshots and mouse/keyboard actions [38]. These capabilities broaden reachable tasks but expand the attack surface for indirect prompt injection and increase the need for sandboxing, permissioning, and human confirmation on sensitive steps.

6.1.2 From Browser to Operating System

Operating-system agents must manage files, switch applications, and recover from partial failures across multi-app workflows. OSWorld benchmarks such end-to-end desktop control and originally highlighted a large gap to human performance due to brittle grounding and long-horizon error accumulation [14]. OSWorld Verified re-evaluates the benchmark to reduce labeling noise and better separate grounding vs. planning errors [100], yielding substantially higher reported performance for top systems; for example, CoAct 1 reports 60.76% success by combining computer-use actions with coding-as-action for verification and recovery [101]. Windows Agent Arena complements this line by focusing on Windows-specific hierarchies and long-horizon reliability [110], reinforcing that progress depends on both base models and surrounding controllers (grounding, verification, recovery) and should be reported with failure categories (unsafe actions, retries, grounding mistakes), not only mean success.

6.1.3 Enterprise and Software Engineering

Enterprise settings emphasize reliability, governance, and extreme horizon lengths. SWE-Bench Pro extends software-agent evaluation toward repository-scale tasks that require hours of simulated work and exposes bottlenecks such as context exhaustion and search-space explosion [13]. For business intelligence,

SQL-agent work demonstrates decomposition of complex questions into multi-stage query plans [111, 112]. However, deployment requires more than academic accuracy: enterprise frameworks emphasize auditability (trace logs), data governance, and failure recovery—dimensions often absent from general benchmarks such as AgentBench [113, 114].

6.2 Embodied Agents: Robotics and Open-Ended Games

Embodied AI represents the challenge of grounding language in physical reality, where actions have irreversible consequences and physics dictates constraints. The field has moved from simple instruction following to Vision-Language-Action (VLA) models that directly map perception to motor control.

6.2.1 Open-Ended Learning in Games

Before tackling physical robots, agents demonstrated complex behaviors in open-ended game environments. Voyager [64] is the flagship example of an agent capable of lifelong learning. Deployed in Minecraft, Voyager writes its own executable Python code to master novel skills (e.g., combat, mining), stores them in a persistent "Skill Library," and retrieves them to compose complex behaviors. Unlike RL agents that require millions of samples, Voyager explores the world via an automatic curriculum, advancing through the game's technology tree 15.3× faster than baselines. This "Code-as-Policy" approach [115] demonstrated that LLMs could reason about long-horizon physical tasks using the abstraction of programming.

6.2.2 Physical Grounding and Affordances

Transferring this reasoning to real robots requires addressing "grounding"—ensuring the agent understands what is physically possible. Early approaches like SayCan [116] used LLMs as high-level planners, filtering semantic plans through a learned value function of robot affordances (i.e., "can I actually pick up this cup?"). This paradigm has evolved into end-to-end VLA models. VLA Architectures [117] catalog over 80 recent models, defining a new taxonomy based on "Multimodal Alignment"—how vision encoders and language tokens are fused directly into action tokens.

6.2.3 VLA Models

The current frontier is represented by systems like Gemini Robotics 1.5 [118]. Unlike Voyager's code generation, Gemini 1.5 introduces "native thinking" for robots, allowing them to process video input and reason internally about a task (e.g., "The cup is fragile, I must grip gently") before generating motor commands. Crucially, it leverages cross-embodiment learning, transferring skills learned on one robot morphology to another, a key step toward general-purpose robotic brains. To further enhance robustness, VLM-GroNav [119] integrates proprioceptive sensing (force/torque feedback) with vision. By grounding VLM outputs in physical feedback, robots can detect hazards (e.g., slippery terrain) that vision alone misses, improving navigation success by 50%.

6.2.4 Autonomous Driving

In the high-stakes domain of Autonomous Driving, agents must reason about rules and social intent. Agent-Driver [120] departs from traditional black-box neural networks by introducing an explicit reasoning engine. The agent uses a "cognitive memory" of traffic rules and a "Chain-of-Thought" planner to explain its decisions (e.g., "Yielding because the pedestrian is entering the crosswalk"), improving safety and interpretability. However, running these massive models on vehicles is computationally prohibitive. To address this latency bottleneck, DiMA [121] proposes a knowledge distillation framework. DiMA compresses giant multimodal models (like GPT-4V) into compact, edge-deployable models, preserving the reasoning logic while reducing parameters by 100×, essential for real-time safety.

6.3 Specialized Domains

While generalist agents attract attention, many near-term impacts are emerging in specialized verticals where agents must integrate with rigorous workflows and meet safety, regulatory, and traceability requirements.

6.3.1 Healthcare and Scientific Discovery

In the natural sciences, agents are shifting from reference tools to research partners. Scientific Intelligence outlines scientific-agent roles spanning hypothesis generation, experiment design, and analysis [9], and full-loop systems demonstrate end-to-end cycles in which agents propose hypotheses, write Python for simulations, and interpret results with minimal human intervention [122]. Surveys also report applications in compound and nanobody discovery, while platforms such as ToolUniverse coordinate access to scientific tools and databases via manager/composer architectures [123, 124].

In clinical settings, the emphasis is precision, patient safety, and legal accountability. Medical agents increasingly connect to EHRs, reason over longitudinal histories, and support clinical decision-making and administrative workflows [125,128]. Agentic Healthcare and MindGuard describe tool-chaining for genomics and imaging and mobile-sensor monitoring for proactive support under audit trails for governance and liability [129, 130]. Correspondingly, evaluation work argues that generic NLP benchmarks are insufficient and calls for medical-specific tests that combine notes with images and verify compliance with professional guidelines [131].

6.3.2 Finance, Economics, and Advanced Conversational Agents

In finance, agentic AI supports both automation and market simulation. PyMarketSim offers a controlled environment where RL and LLM agents trade in realistic order books [132], and financial-reasoning studies suggest heterogeneous agent behaviors can reproduce phenomena such as liquidity provision, microstructure exploitation, and bubble-like dynamics, motivating agent-based simulation as a tool for macroeconomic research [133].

Conversational agents optimize for engagement and social interaction rather than profit. Proactive conversational AI spans reactive systems through deliberative agents that introduce topics and pursue long-term objectives (e.g., education, sales) [134]. Empathetic voice agents combine speech generation with affect modeling so tone and pacing adapt to users, and long-term support agents aim to reduce loneliness via sustained, reliable interactions [135, 136]. Because real-time dialogue is latency sensitive (even sub-second delays degrade presence), deployments emphasize aggressive optimization such as distillation and speculative decoding [137].

7 Evaluation and Safety

As agents transition from closed sandboxes to real world deployment evaluation methodologies must evolve beyond simple text similarity metrics like BLEU or ROUGE. Integrating performance metrics [4] with enterprise deployment requirements [113], we adopt the CLASSic framework [75] to assess the field in five critical dimensions including Cost, Latency, Accuracy, Security, and Stability.

7.1 Cost: The Efficiency Trade Off

High reasoning depth often comes at the price of significant computational overhead [71]. As illustrated in Fig. 4 hierarchical architectures such as ReAcTree maximize task proficiency and reasoning depth but they incur exponential increases in token consumption [70] compared to standard linear chains or zero shot prompting. This Efficiency Intelligence Trade off remains a critical bottleneck for deploying agents in cost sensitive real time applications.

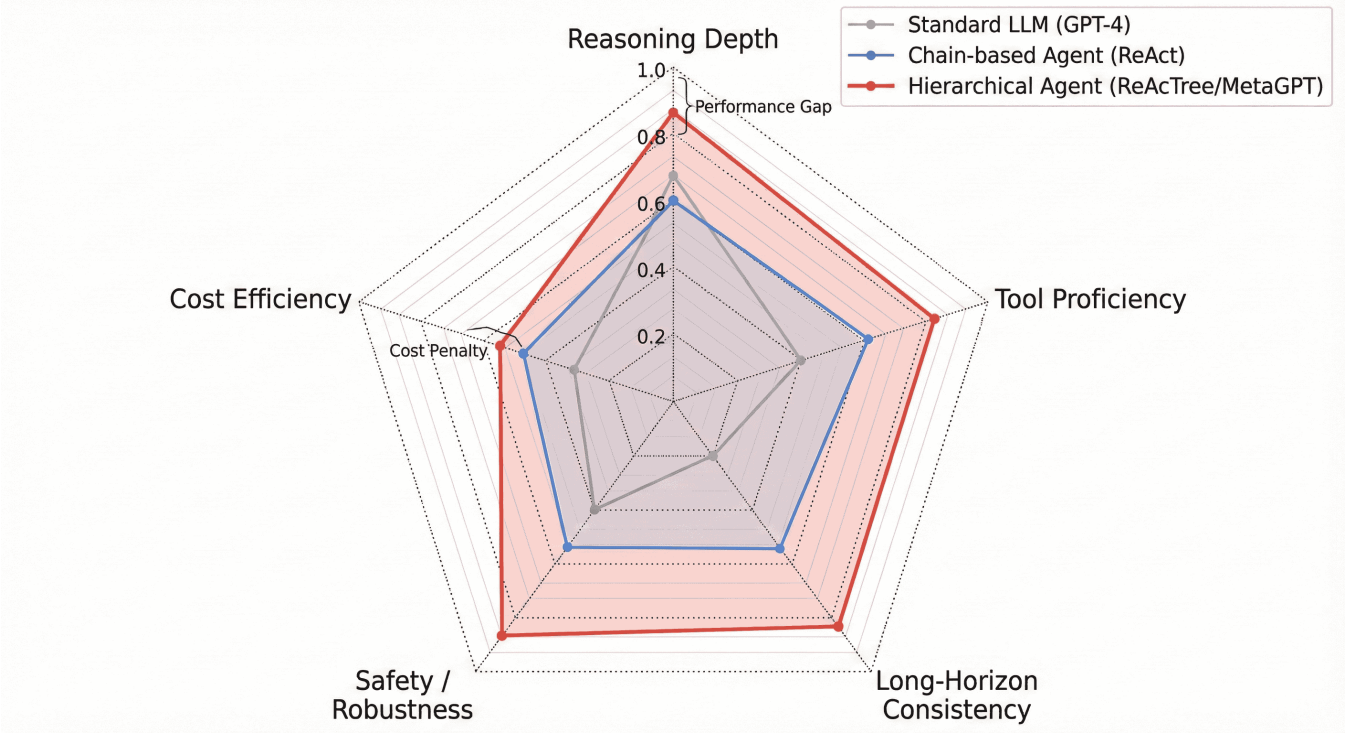

Figure 4: **Multidimensional Architectural Comparison.** We compare architectures across the CLASSic dimensions. While Hierarchical Agents (Red) achieve superior Reasoning Depth and Tool Proficiency, they incur a significant Cost Penalty and Latency compared to standard LLMs (Grey).

7.2 Latency: Real World Constraints

Real world tasks are rarely instantaneous necessitating rigorous latency evaluation. Asynchronous benchmarks like Robotouille [138] reveal that agents frequently fail when tasks involve variable temporal delays such as waiting for a cooking process to finish before acting. The study shows that while synchronous agents achieve 47% success asynchronous settings cause performance to plummet to 11% highlighting a critical lack of temporal awareness in current planners. To mitigate latency in safety critical domains like autonomous driving where sub 100ms response times are mandatory DiMA [121] proposes knowledge distillation. This approach compresses giant multimodal planners like GPT-4V into compact edge deployable models preserving the reasoning logic while reducing parameter count by 100 times thereby solving the latency constraint without sacrificing safety logic.

7.3 Accuracy: The capability gap and saturation

Accuracy for agents is not captured by static question answering alone: success can collapse when tasks require tool use, state tracking, and long-horizon recovery, reflecting both base-model limits and end-to-end architecture choices (memory, orchestration, grounding, and permissions). GAIA highlights this gap for general assistants on human-easy tasks that require multi-step decomposition, tool use, and verification [39]. For desktop control, OSWorld Verified reduces evaluation noise and exposes stronger performance under more reliable protocols [100]; CoAct 1 exemplifies a modern design that combines computer-use actions with coding-as-action to validate and repair steps during execution [101]. In software engineering, SWE-bench Verified reduces ambiguity and brittle tests [40,41], while SWE-bench Pro pushes toward repo-scale, longer-horizon tasks where context management and search control become central bottlenecks [13]. For tool use under policies, τ-bench evaluates multi-turn interaction in domains like retail and airlines [43]. FrontierMath targets hard-ceiling mathematical reasoning with reduced contamination, stressing inference-

time reasoning and verifier-guided approaches [42].

Consequently, modern agent evaluation increasingly reports not only mean success but also compute budgets, run-to-run variance, and failure severity distributions, aligning accuracy with CLASSic trade-offs in realistic trajectories. AgentBench evaluates agents across diverse environments [114], while MultiAgentBench (MARBLE) measures emergent multi-agent behaviors such as negotiation efficiency and consensus formation [82].

7.4 Security: The trust gap and prompt injection

Security is a primary barrier to deploying agentic systems: once an LLM is connected to executable tools (file I/O, code execution, enterprise APIs), prompt injection can override the intended objective and turn the agent into a confused deputy [139]. Attacks may be direct (user input) or indirect via untrusted content encountered during tool use (web pages, documents, tickets, tool outputs). The risk is amplified in high-bandwidth observation channels such as computer-use settings, where agents interpret screenshots and execute low-level mouse/keyboard actions [37, 38]. Standardized connector layers (e.g., MCP) further increase the need for governance at integration boundaries, including allowlists, authentication, and audit logging [34, 35].

Prompt-only defenses are brittle: PromptArmor reduces the likelihood that injected instructions are followed [140], but adaptive attackers can craft indirect injections that bypass static guards [141]. As a result, robust security is increasingly a systems problem: layered mitigations such as constrained tool permissions, compartmentalized sandboxes, explicit user confirmation for sensitive actions, and independent policy/audit components that validate plans before execution, plus operational monitoring and intervention to limit the impact of unsafe behavior in production [143].

7.5 Stability: Failure Mode Analysis

Finally Stability refers to the system variance over repeated runs [73] and its resilience to minor perturbations. In stochastic systems like LLM agents a simple Success Rate metric often masks critical reliability issues. A framework for enterprise agents [113] argues that rigorous evaluation must include failure mode analysis quantifying not just how often an agent succeeds but the severity distribution of its failures distinguishing a benign timeout from a catastrophic data leak. This is particularly vital in regulated domains. In healthcare applications [131] emphasizes that agents must demonstrate high compliance stability ensuring that clinical decisions consistently align with medical guidelines regardless of prompt phrasing or stochastic sampling temperature. Future benchmarks must therefore report standard deviation and worst case failure scenarios alongside mean performance to provide a true picture of agent readiness.

8 Challenges and Future Directions

Despite the rapid proliferation of agentic architectures, the field remains in a nascent stage. While agents can perform impressive feats in controlled sandboxes, their deployment in unconstrained real-world environments is hindered by fundamental limitations in reliability, efficiency, and alignment. In this section, we synthesize the critical open challenges and outline promising directions for future research.

8.1 Hallucination in Action and Error Propagation

The most pervasive challenge in agentic AI is the "hallucination in action" problem. While a factual error in a chatbot response is merely misleading, a hallucinated action—such as calling a non-existent API endpoint or deleting the wrong file—can lead to irreversible system failures. Agents often fail when the retrieval component provides irrelevant context [144] the retrieval component provides irrelevant context, causing the planner to generate flawed execution steps. Furthermore, in multi-step reasoning loops like ReAct, a single error in an early step propagates downstream, leading to "cascading failures." Future work

must focus on robust error-recovery mechanisms, potentially leveraging techniques like SelfCheckGPT [145] to validate reasoning steps before execution occurs.

8.2 Infinite Loops and Agent Paralysis

Autonomous agents frequently suffer from getting stuck in repetitive loops, continuously retrying a failed action without modifying their strategy. Benchmarks like WebArena report low success rates (often ¡15%) on long-horizon tasks, partially due to agents failing to recognize when they are caught in a local optimum. While architectures like Reflexion attempt to mitigate this via verbal feedback, agents still struggle with "giving up" or asking for human help appropriately. Developing "meta-cognitive" modules that allow an agent to assess its own progress and interrupt futile loops is a critical research direction.

8.3 Latency and Computational Cost

The transition from single-agent to multi-agent systems has introduced significant computational overhead. Architectures that rely on extensive debate or tree-search, such as ToT [26], require multiple LLM inference calls for a single user query. This latency is unacceptable for real-time applications. There is a pressing need for 'System 2' thinking to be distilled into efficient 'System 1' reflexes [10]. Research into ReWOO [146] offers a path forward by separating planning from execution, but further optimization is required to make agentic AI economically viable at scale.

8.4 Human-Agent Alignment and Social Norms

As agents become more autonomous, ensuring they adhere to human values and social norms becomes paramount. An agent optimized solely for task completion might behave ruthlessly—for example, spamming a user's contact list to achieve a "networking" goal. Recent work on socially aligned agents [148] argues that agents must be trained not just on task success, but on adherence to social contracts and safety constraints. This involves moving beyond simple Reinforcement Learning from Human Feedback (RLHF) toward constitutional AI frameworks where agents intrinsically respect boundaries without needing explicit instructions for every edge case.

8.5 Towards Open-Ended Learning

Current agents are largely static; they do not evolve their core competencies after deployment. A major frontier is the development of agents capable of open-ended self-improvement. OMNI [151] proposes systems that generate their own curricula, seeking out novel tasks to expand their skill capabilities. This aligns with the vision of Voyager [64], suggesting a future where agents operate as lifelong learners, continuously acquiring, refining, and sharing new skills without human intervention.

8.6 Theoretical Limits and Optimization

Despite empirical success, the theoretical boundaries of agentic AI remain understudied. OMNI [151] suggests that for agents to be truly autonomous, they must possess an intrinsic motivation function to generate their own curriculum, a feature currently absent in standard LLMs. Furthermore, optimization remains a bottleneck; TALM [150] and work on active retrieval [144] highlight the latency costs of iterative retrieval. Future architectures may need to integrate introspective capabilities [149] to balance the trade-off between expensive external tool calls and cheaper internal introspection.

9 Conclusions

Our investigation on Agentic AI landscape suggests that the central design question is shifting from how to prompt a model to how to program and control a complete agent system. This inversion appears most

clearly along three dimensions:

1. **Reasoning:** Architectures have moved from myopic single loop solvers such as ReAct to hierarchical and search based systems, and increasingly to reasoning models that internalize parts of search and backtracking at inference time under a controllable compute budget.

2. **Action:** The interaction paradigm has expanded from constrained API calls to code as action and computer use actions, enabling agents to operate both structured tool APIs and arbitrary graphical interfaces, with verification and recovery becoming first class design requirements.

3. **Collaboration:** Multi agent systems are moving away from unstructured chat loops toward controllable workflow graphs and explicit handoff patterns, improving observability, debuggability, and safety through flow engineering.

At the same time, the transition from chatbot to reliable agent is incomplete. Recent verified evaluations show rapid gains in desktop and operating system control, but they also make clear that success depends strongly on the surrounding controller, including grounding, tool permissions, and recovery loops [100,101]. Closing the remaining gap to human level reliability will require progress on two coupled fronts: i) efficient reasoning and verification that improves robustness without prohibitive cost or latency, and ii) stronger security and governance that can withstand indirect prompt injection and other confused deputy failures when agents are connected to real tools and data.

Ultimately, progress in Agentic AI is unlikely to come from model scale alone. It will depend on architectures that integrate perception, memory, tools, and collaboration in a way that is not only powerful, but also controllable, auditable, and aligned with the constraints of real world deployment.

References

[1] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2023.

[2] J. W. Choi *et al.*, "ReAcTree: Hierarchical LLM Agent Trees with Control Flow for Long-Horizon Task Planning," *arXiv preprint arXiv:2511.02424*, 2025.

[3] X. Wang *et al.*, "Executable Code Actions Elicit Better LLM Agents," *International Conference on Machine Learning (ICML)*, 2024.

[4] A. Khamis, M. Elshakankiri, & H. Elsayed, "Agentic AI Systems: Architecture and Evaluation Using a Frictionless Parking Scenario," *IEEE Access*, vol. 13, p. 11083588, 2025.

[5] M. Abou Ali, F. Dornaika, & J. Charafeddine, "Agentic AI: a comprehensive survey of architectures, applications, and future directions," *Artificial Intelligence Review*, vol. 59, no. 1, p. 11, 2025.

[6] N. R. Jennings, "On agent-based software engineering," *Artificial Intelligence*, vol. 117, no. 2, pp. 277–296, 2000.

[7] F. Piccialli *et al.*, "AgentAI: A Comprehensive Survey on Autonomous Agents in Distributed AI for Industry 4.0," *Expert Systems with Applications*, p. 128404, 2025.

[8] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[9] Y. Ren *et al.*, "Towards Scientific Intelligence: A Survey of LLM-based Scientific Agents," *arXiv preprint arXiv:2411.13837*, 2025.

[10] Y. Li *et al.*, "Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security," *arXiv preprint arXiv:2401.05459*, 2024.

[11] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *International Conference on Learning Representations (ICLR)*, 2024.

[12] Y. Yang *et al.*, "Navigating the Risks: A Survey of Security, Privacy, and Ethics Threats in LLM-Based Agents," *arXiv preprint arXiv:2411.09523*, 2024.

[13] X. Deng *et al.*, "SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?," *arXiv preprint arXiv:2509.16941*, 2025.

[14] T. Xie *et al.*, "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments," *arXiv preprint arXiv:2404.07972*, 2025.

[15] L. Busoniu, R. Babuska, & B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics*, 2008.

[16] G. Mialon *et al.*, "Augmented Language Models: a Survey," *Transactions on Machine Learning Research*, 2023.

[17] P. Li *et al.*, "A Review of Prominent Paradigms for LLM-Based Agents: Tool Use (Including RAG), Planning, and Feedback Learning," *arXiv preprint arXiv:2406.05804*, 2025.

[18] L. Wang *et al.*, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024.

[19] S. Hao *et al.*, "Reasoning with Language Model is Planning with World Model," *arXiv preprint arXiv:2305.14992*, 2023.

[20] A. Li *et al.*, "Agent-oriented planning in multi-agent systems," *arXiv preprint arXiv:2410.02189*, 2024.

[21] A. Kumar, "Building Autonomous AI Agents based AI Infrastructure," *International Journal of Computer Trends and Technology*, vol. 72, no. 11, pp. 116–125, 2024.

[22] J. Luo *et al.*, "Large language model agent: A survey on methodology, applications and challenges," *arXiv preprint arXiv:2503.21460*, 2025.

[23] Y. Cheng *et al.*, "Exploring Large Language Model based Intelligent Agents," *arXiv preprint arXiv:2403.00000*, 2024.

[24] Y. Qu *et al.*, "Tool Learning with Large Language Models: A Survey," *arXiv preprint arXiv:2304.08354*, 2025.

[25] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[26] S. Yao *et al.*, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[27] Y. Chen *et al.*, "GoalAct: Enhancing LLM-Based Agents via Global Planning and Hierarchical Execution," *arXiv preprint arXiv:2410.11964*, 2025.

[28] OpenAI, "Learning to reason with LLMs," 2024. [Online]. Available: https://openai.com/index/learning-to-reason-with-llms/

[29] OpenAI, "o1 system card," 2024. [Online]. Available: https://openai.com/index/openai-o1-system-card/

[30] OpenAI, "OpenAI o3 and o4-mini system card," 2025. [Online]. Available: https://openai.com/index/o3-o4-mini-system-card/

[31] C. Snell, J. Lee, K. Xu, & A. Kumar, "Scaling LLM test time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[32] LangChain AI, "LangGraph: building language agents as graphs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[33] OpenAI, "Swarm," 2024. [Online]. Available: https://github.com/openai/swarm

[34] Anthropic, "Introducing the Model Context Protocol," 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[35] Anthropic, "Model Context Protocol specification," 2024. [Online]. Available: https://modelcontextprotocol.io/specification

[36] Anthropic, "Developing a computer use model," 2024. [Online]. Available: https://www.anthropic.com/news/developing-computer-use

[37] Anthropic, "Computer use tool documentation," [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool

[38] OpenAI, "Introducing Operator," 2025. [Online]. Available: https://openai.com/index/introducing-operator/

[39] G. Mialon *et al.*, "GAIA: a benchmark for general AI assistants," *arXiv preprint arXiv:2311.12983*, 2023.

[40] OpenAI, "Introducing SWE bench Verified," 2024. [Online]. Available: https://openai.com/index/introducing-swe-bench-verified/

[41] SWE bench, "SWE bench leaderboards," 2024. [Online]. Available: https://www.swebench.com/

[42] Epoch AI, "FrontierMath: a benchmark for evaluating advanced mathematical reasoning in AI," *arXiv preprint arXiv:2411.04872*, 2024.

[43] J. Wu *et al.*, "τ-bench: a benchmark for tool agent user interaction in real world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[44] N. Shinn *et al.*, "Reflexion: Language Agents with Verbal Reinforcement Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[45] R. Aksitov *et al.*, "REST Meets ReAct: Self-Improvement for Multi-Step Reasoning LLM Agent," *arXiv preprint arXiv:2312.10003*, 2024.

[46] Z. Zhou *et al.*, "AgentCoMa: A Compositional Benchmark Mixing Commonsense and Mathematical Reasoning," *arXiv preprint arXiv:2508.19988*, 2025.

[47] H. Furuta *et al.*, "Language Model Agents Suffer from Compositional Decision Making," *arXiv preprint arXiv:2410.15037*, 2024.

[48] Y. Sun, H. Wang, Y. Yao, & Z. Gong, "Can LLM-Reasoning Models Replace Classical Planning?," *arXiv preprint arXiv:2412.10395*, 2025.

[49] H. Qu & T. Xie, "Teaching Language Model Agents How to Self-Improve," *arXiv preprint arXiv:2410.12468*, 2024.

[50] E. Acikgoz *et al.*, "Self-Improving LLM Agents at Test-Time," *arXiv preprint arXiv:2412.10029*, 2025.

[51] Z. Liu *et al.*, "Proactive Conversational Agents with Inner Thoughts and Empathy," *arXiv preprint arXiv:2405.19464*, 2024.

[52] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[53] D. Hendrycks *et al.*, "Measuring Massive Multitask Language Understanding," *International Conference on Learning Representations (ICLR)*, 2021.

[54] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.

[55] Z. Yang *et al.*, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," *EMNLP*, 2018.

[56] M. Shridhar *et al.*, "ALFWorld: Aligning Text and Embodied Environments for Interactive Learning," *International Conference on Learning Representations (ICLR)*, 2020.

[57] S. G. Patil, T. Zhang, X. Wang, & J. E. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[58] A. Vuddanti *et al.*, "PALADIN: Self-Correcting Language Model Agents to Cure Tool-Failure Cases," *arXiv preprint arXiv:2308.05201*, 2025.

[59] J. S. Park *et al.*, "Generative Agents: Interactive Simulacra of Human Behavior," *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[60] S. Hong *et al.*, "Enhancing Memory Retrieval in Generative Agents through LLM-Trained Cross Attention Networks," *arXiv preprint arXiv:2410.15693*, 2025.

[61] W. Zhong *et al.*, "MemoryBank: Enhancing Large Language Models with Long-Term Memory," *arXiv preprint arXiv:2305.10250*, 2024.

[62] C. Hu *et al.*, "ChatDB: Augmenting LLMs with Databases as their Symbolic Memory," *arXiv preprint arXiv:2306.03901*, 2024.

[63] Z. Xi *et al.*, "The Rise and Potential of Large Language Model Based Agents: A Survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.

[64] G. Wang *et al.*, "Voyager: An Open-Ended Embodied Agent with Large Language Models," *Transactions on Machine Learning Research (TMLR)*, 2024.

[65] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[66] A. Patel *et al.*, "Large language models can self-improve at web agent tasks," *arXiv preprint arXiv:2405.20309*, 2024.

[67] Z. Chen *et al.*, "Agent-FLAN: Designing data and methods of effective agent tuning for large language models," *arXiv preprint arXiv:2403.12881*, 2024.

[68] Y. Xia *et al.*, "AgentRM: Enhancing agent generalization with reward modeling," *arXiv preprint arXiv:2502.18407*, 2025.

[69] Z. Xi *et al.*, "AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress," *arXiv preprint arXiv:2511.08325*, 2025.

[70] AI CERTs Team, "RE-Bench: Economic Efficiency Factors in Agent Evaluation," *AI CERTs Technical Report*, 2025.

[71] B. Cottier *et al.*, "LLM Inference Prices Have Fallen Rapidly but Unequally Across Tasks," *Epoch AI*, 2025.

[72] T. Zhang *et al.*, "When Hallucination Costs Millions: Benchmarking AI Agents in High-Stakes Adversarial Financial Markets," *arXiv preprint*, 2025.

[73] Scale AI Research, "Smoothing Out LLM Variance for Reliable Enterprise Evals," *Scale AI Technical Blog*, 2025.

[74] A. Zhao *et al.*, "Expel: LLM Agents are Experiential Learners," *arXiv preprint arXiv:2308.10144*, 2023.

[75] M. Wornow *et al.*, "Top of the CLASS: Benchmarking LLM Agents on Real-World Enterprise Tasks," *ICLR 2025 Workshop on Trustworthy LLMs*, 2025.

[76] X. Zhao *et al.*, "MemInsight: Autonomous Memory Augmentation for LLM Agents," *arXiv preprint arXiv:2503.21760*, 2025.

[77] Z. Ma *et al.*, "MemAgent: Reshaping Long-Context LLM with Multi-Conversation Reinforcement Learning," *arXiv preprint arXiv:2507.02259*, 2025.

[78] Y. Ge *et al.*, "Chain-of-Agents: Large Language Models Collaborating on Long-Context Tasks," *arXiv preprint arXiv:2406.02818*, 2025.

[79] G. Li *et al.*, "CAMEL: Communicative Agents for 'Mind' Exploration," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[80] C. Wang *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," *Conference on Language Modeling (COLM)*, 2024.

[81] H. Tran *et al.*, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2025.

[82] X. Zhu *et al.*, "MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents," *arXiv preprint arXiv:2405.16960*, 2025.

[83] Y. Zhang *et al.*, "Towards Efficient LLM Grounding for Embodied Multi-Agent Collaboration," *arXiv preprint arXiv:2405.14314*, 2025.

[84] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," *arXiv preprint arXiv:2307.07924*, 2024.

[85] E. Meyerson *et al.*, "Solving a Million-Step LLM Task with Zero Errors," *arXiv preprint arXiv:2511.09030*, 2025.

[86] Z. Liu *et al.*, "BOLAA: Benchmarking and Orchestrating LLM-Augmented Autonomous Agents," *arXiv preprint arXiv:2308.05960*, 2024.

[87] Y. Xiao, Y. Cao, H. Zhang, & W. Liu, "TradingAgents: Multi-Agent LLM Financial Trading Framework," *arXiv preprint arXiv:2409.12857*, 2024.

[88] D. Gosmar & D. A. Dahl, "Hallucination Mitigation with Agentic AI NLP-Based Frameworks," *Available at SSRN 5086241*, 2025.

[89] B. Kwartler, Y. Zhang, Z. Liu, & H. Wang, "Good Parenting is All You Need: Multi-Agentic LLM Hallucination Mitigation," *arXiv preprint arXiv:2410.14262*, 2024.

[90] Y. Du *et al.*, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," *arXiv preprint arXiv:2305.14325*, 2024.

[91] C. Zhang *et al.*, "AppAgent: Multimodal Agents as Smartphone Users," *arXiv preprint arXiv:2312.13771*, 2024.

[92] B. Zheng *et al.*, "GPT-4V(ision) is a Generalist Web Agent, if Grounded," *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[93] R. Huang *et al.*, "AudioGPT: Understanding and Generating Speech, Music, Sound, and Talking Head," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 21, 2024.

[94] Y. Hong *et al.*, "3D-LLM: Injecting the 3D World into Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.

[95] Y. Zhang *et al.*, "Adaptive Heterogeneous Multi-Agent Debate for Large Language Models," *arXiv preprint arXiv:2410.13456*, 2025.

[96] K. M. Jablonka, P. Schwaller, A. Ortega-Guerrero, & B. Smit, "Compositional Communication with LLMs and Reasoning about Chemical Structures," *Nature Machine Intelligence*, 2024.

[97] OpenAI, "GPT-5.1 Codex Max system card," 2025. [Online]. Available: https://openai.com/index/gpt-5-1-codex-max-system-card/

[98] Google, "Introducing Gemini 3: Our most capable model family yet," 2025. [Online]. Available: https://blog.google/technology/google-deepmind/gemini-3/

[99] Anthropic, "Claude Haiku 4.5 system card," 2025. [Online]. Available: https://assets.anthropic.com/m/37cf170ec9d01f5e/original/Claude-Haiku-4-5-System-Card.pdf

[100] XLANG Lab, "Introducing OSWorld Verified," 2025. [Online]. Available: https://xlang.ai/blog/osworld-verified

[101] X. Xin *et al.*, "CoAct 1: Computer using agents with coding as actions," 2025. [Online]. Available: https://linxins.net/coact/

[102] C. Xu *et al.*, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[103] Y. Zhang *et al.*, "SocioVerse: A World Model for Social Simulation via Multi-Agent Systems," *arXiv preprint arXiv:2410.14567*, 2025.

[104] DeepMind Team, "Genie 3: A New Frontier for World Models," *Technical Report*, 2025.

[105] Z. Zhang *et al.*, "PAN: A World Model for General, Interactable, and Long-Horizon Agents," *arXiv preprint arXiv:2410.15678*, 2025.

[106] X. Deng *et al.*, "Mind2Web: Towards a Generalist Agent for the Web," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[107] S. Zhou *et al.*, "WebArena: A Realistic Web Environment for Autonomous Agents," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[108] H. He *et al.*, "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models," *arXiv preprint arXiv:2401.13919*, 2024.

[109] K. Ma *et al.*, "Multimodal LLM Agents are Susceptible to Environmental Distractions," *arXiv preprint arXiv:2410.14890*, 2025.

[110] R. Wu *et al.*, "Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale," *arXiv preprint arXiv:2409.08264*, 2025.

[111] Y. Chen *et al.*, "LLM-Powered SQL Agents for BI & Data Analytics," *arXiv preprint arXiv:2408.06259*, 2025.

[112] M. Chernyshevich *et al.*, "Multi-hop LLM Agent for Tabular Question Answering," *arXiv preprint arXiv:2405.13678*, 2025.

[113] Y. Wang *et al.*, "A Multi-Dimensional Framework for Evaluating Enterprise AI Agents," *arXiv preprint arXiv:2410.15890*, 2025.

[114] X. Liu *et al.*, "AgentBench: Evaluating LLMs as Agents," *International Conference on Learning Representations (ICLR)*, 2024.

[115] J. Liang *et al.*, "Code as Policies: Language Model Programs for Embodied Control," *International Conference on Robotics and Automation (ICRA)*, 2023.

[116] M. Ahn *et al.*, "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," *Conference on Robot Learning (CoRL)*, 2022.

[117] S. Din *et al.*, "Vision Language Action Models in Robotic Manipulation: A Systematic Review," *arXiv preprint arXiv:2408.14993*, 2025.

[118] Google DeepMind Team, "Gemini Robotics 1.5: AI Agents into the Physical World," *Technical Report*, 2025.

[119] M. Elnoor *et al.*, "Robot Navigation Using Physically Grounded Vision-Language Models," *arXiv preprint arXiv:2407.14845*, 2024.

[120] J. Mao *et al.*, "Agent-Driver: A Language Agent for Autonomous Driving," *arXiv preprint arXiv:2311.01135*, 2024.

[121] S. Hegde *et al.*, "Distilling Multi-modal Large Language Models for Autonomous Driving," *arXiv preprint arXiv:2410.16234*, 2025.

[122] D. Zhou *et al.*, "Autonomous Agents for Scientific Discovery," *arXiv preprint arXiv:2410.13567*, 2025.

[123] T. So *et al.*, "Scientific Discoveries by LLM Agents," *Nature*, 2025.

[124] Kempner Institute *et al.*, "ToolUniverse: Building AI Agents for Science," *arXiv preprint arXiv:2410.14678*, 2025.

[125] S. Wang *et al.*, "A Survey of LLM-based Agents in Medicine," *arXiv preprint arXiv:2404.11585*, 2025.

[126] Z. Liu *et al.*, "Dynamic LLM-Agent Network: An LLM-agent Collaboration Framework with Agent Team Optimization," *Proceedings of the Conference on Language Modeling (COLM)*, 2024.

[127] A. Zhou *et al.*, "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models," *International Conference on Machine Learning (ICML)*, 2024.

[128] Nature Digital Medicine *et al.*, "Healthcare Agent: Eliciting the Power of LLMs," *Nature Digital Medicine*, 2025.

[129] K. Yuan *et al.*, "Agentic Large Language Models for Healthcare," *arXiv preprint arXiv:2410.15890*, 2025.

[130] ACM *et al.*, "MindGuard: Autonomous LLM Agent for Mental Health Using Mobile Sensor Data," *ACM Proceedings*, 2025.

[131] Y. Chen *et al.*, "Evaluating Large Language Models and Agents in Healthcare," *arXiv preprint arXiv:2409.14567*, 2025.

[132] A. Mascioli, Z. Li, H. Zhang, & Y. Wang, "A Financial Market Simulation Environment for Trading Agents," *ICAIF*, 2024.

[133] A. Lopez-Lira *et al.*, "Can Large Language Models Trade? Testing Financial Reasoning," *arXiv preprint arXiv:2408.14234*, 2025.

[134] ACM *et al.*, "Proactive Conversational AI: A Comprehensive Survey," *arXiv preprint arXiv:2405.13987*, 2025.

[135] IJISAE *et al.*, "Empathetic Intelligence: LLM-Based Conversational AI Voice Agent," *IJISAE*, 2024.

[136] S. Alotaibi *et al.*, "The Role of Conversational AI Agents in Providing Support for Isolated Individuals," *arXiv preprint arXiv:2410.14890*, 2024.

[137] RJPN *et al.*, "Generative AI for Real-Time Conversational Agents," *arXiv preprint arXiv:2410.15234*, 2024.

[138] G. Gonzalez-Pumariega *et al.*, "Robotouille: An Asynchronous Planning Benchmark for LLM Agents," *arXiv preprint arXiv:2502.05227*, 2025.

[139] Y. Liu *et al.*, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," *arXiv preprint arXiv:2310.12815*, 2023.

[140] Y. Chen *et al.*, "PromptArmor: Simple yet Effective Prompt Injection Defenses," *arXiv preprint arXiv:2507.15219*, 2025.

[141] H. Zhan *et al.*, "Adaptive Attacks Break Defenses Against Indirect Prompt Injection," *arXiv preprint arXiv:2503.00061*, 2025.

[142] Y. Zhang *et al.*, "Mitigating Spatial Hallucination in LLMs," *arXiv preprint arXiv:2410.13567*, 2025.

[143] AWS, "Reducing hallucinations in large language models with custom intervention using Amazon Bedrock Agents," 2024.

[144] Z. Jiang *et al.*, "Active Retrieval-Augmented Generation for Knowledge-Intensive NLP," *arXiv preprint arXiv:2305.06983*, 2023.

[145] P. Manakul, A. Liusie, & M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection," *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[146] B. Xu *et al.*, "ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models," *arXiv preprint arXiv:2305.18323*, 2023.

[147] Y. Zhang *et al.*, "Personal Large Language Model Agents: A Case Study on Tailored Travel Planning," *arXiv preprint arXiv:2401.05459*, 2024.

[148] I. Gabriel *et al.*, "Socially Aligned Agents," *arXiv preprint arXiv:2404.13789*, 2024.

[149] L. Chen *et al.*, "Introspective Tips: Large Language Models with Self-Reflection," *arXiv preprint arXiv:2305.11598*, 2023.

[150] A. Parisi, Y. Zhao, & N. Fiedel, "TALM: Tool Augmented Language Models," *arXiv preprint arXiv:2205.12255*, 2022.

[151] J. Zhang *et al.*, "OMNI: Open-endedness via Models of human Notions of Interestingness," *arXiv preprint arXiv:2306.01711*, 2023.

[152] Contributors, "The Ultimate Guide to Fine-Tuning LLMs from Basics," *Technical Report*, 2024.

[153] D. M. Anisuzzaman *et al.*, "Fine-tuning large language models for specialized use cases," *Mayo Clinic Proceedings: Digital Health*, vol. 3, no. 1, p. 100184, 2025.

[154] OpenAI, "OpenAI Practical Guide to Building Agents," 2025.

[155] Z. Gou *et al.*, "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing," *International Conference on Learning Representations (ICLR)*, 2024.

[156] J. Yang *et al.*, "Magma: A Foundation Model for Multimodal AI Agents," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[157] V. Pavlyshyn, R. Gomez, & S. Li, "Time-aware Knowledge Graphs for Episodic Memory in LLM Agents," *arXiv preprint arXiv:2501.00987*, 2025.

[158] B. Chen *et al.*, "FireAct: Toward Language Agent Fine-tuning," *International Conference on Learning Representations (ICLR)*, 2024.

[159] H. Lee *et al.*, "RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback," *arXiv preprint arXiv:2309.00267*, 2024.